Передача в локальную сессию Claude Code (Windows, машина пользователя)

Этот документ — задание для сессии Claude Code, запущенной **на компьютере пользователя** в папке репозитория (актуальная ветка — **`claude/busy-darwin-juot7g`**: в ней всё из `claude/optimistic-ride-hu7e74` плюс доработки облачной сессии от 2026-10-16, см. раздел 8; например `C:\Users\Stryker\Downloads\LOCAL_AGENT_PROJECT-claude-optimistic-ride-hu7e74` после `git fetch`, или свежий `git clone`). Облачная сессия (Linux, без GPU, без Windows SDK, без реестра Ollama) довела код до зелёных проверок, но всё, что требует Windows, видеокарты, живой модели, Godot и ComfyUI, проверяется только здесь. Выполняй разделы по порядку, каждый заканчивается критерием «готово».

Правила проекта — в `AGENTS.md` (отвечать по-русски, код и имена по-английски, оболочка — Windows PowerShell 5.1: без `&&`, без grep/sed; Godot 4.8 лежит в `F:\PROJECTS\godot-hd\engine\bin\godot.windows.editor.x86_64.exe`).

Как запустить локальную сессию. В PowerShell перейти в папку репозитория, выполнить `git fetch origin claude/busy-darwin-juot7g` и `git checkout claude/busy-darwin-juot7g` (в скачанном zip распакована более старая ветка `claude/optimistic-ride-hu7e74` — нужен `git fetch`), затем `claude` и первым сообщением: «Прочитай `docs/HANDOFF_LOCAL_SESSION.md` и выполняй разделы 1–9 по порядку, после каждого раздела коротко отчитывайся». PDF-версия этого файла (`docs/HANDOFF_LOCAL_SESSION.pdf`) — копия для чтения, источник истины — Markdown в репозитории.

0. Что уже сделано (не переделывать)

Компонент	Где	Проверено в облаке
Десктоп-клиент CodeWhale (Tauri 2 + React)	`codewhale-desktop/`	tsc, vite build, 41 e2e (моки IPC, включая менеджер моделей), clippy, 47 Rust-тестов; CI на `windows-latest` зелёный
MCP-сервер `ai-drive` (диск знаний)	`src-tauri/src/drive_mcp.rs`, команды `src-tauri/src/drive.rs`, панель `src/components/drive/`	сквозной тест с настоящим CodeWhale + Godot 4.7.2 headless (`docs/FINAL_TEST_WATER.md`, `docs/final-test/water.png`)
MCP-сервер `comfy` (фабрика ассетов)	`src-tauri/src/comfy_mcp.rs`, `src-tauri/src/comfy.rs`, панель `src/components/comfy/`	против стенда `mock_comfy.py` с формами API ComfyUI v0.11 (`final_test_comfy.py`, 17/17)
Скиллы, шаблоны, скрипты	`src-tauri/resources/` (skills/, comfy/, drive/, godot-check/)	синтаксис, юнит-тесты
Демо-проект воды	`Godot/WaterDemo/`	шейдер компилируется, рендер под Xvfb
Установщик/portable	`.github/workflows/release.yml` (NSIS setup, MSI, portable zip)	релиз v0.1.0 опубликован, SHA-256 трёх файлов сверены — см. раздел 7
Документация	`docs/INSTALL.md`, `docs/KNOWLEDGE_DRIVE.md`, `docs/COMFYUI.md`,	—

Компонент	Где	Проверено в облаке
	docs/FINAL_TEST_WATER.md, PROJECT.md	

Исследовательские материалы агентов лежат в `docs/research/` (JSON/MD); их синтез готов (2026-10-16): `codewhale-safety-design.md` (модель безопасности), `rtx5060-8gb-model-plan.md` (план моделей под RTX 5060 8 ГБ), `user-pdf-verification.md` (верификация PDF пользователя). Перед разделами 3–5 полезно прочитать план моделей: какую модель и с каким `num_ctx` брать.

1. Проверка окружения (10 минут)

Выполни и запиши результат каждой команды:

```
        node -v            # нужен 22.x
        npm -v
        rustc -vV          # host: x86_64-pc-windows-msvc; если нет rustup — https://rustup.rs, toolchain stable-msvc
        cargo --version
        ollama --version   # Ollama установлена и запущена (трей); иначе https://ollama.com/download
        ollama list        # хотя бы одна модель с вызовом инструментов, напр. qwen2.5-coder:7b
        python --version   # Python 3.11+; затем: python -c "import sympy; print(sympy.__version__)"
        git --version
        Test-Path "F:\PROJECTS\godot-hd\engine\bin\godot.windows.editor.x86_64.exe"
        Get-AppxPackage -Name "*WebView2*" | Select-Object Name, Version   # пусто — установить Evergreen WebView2
        nvidia-smi --query-gpu=name,memory.total --format=csv
```

Нет Visual Studio Build Tools (C++ workload, Windows SDK) — поставить: без них не соберётся `rusqlite` (SQLite компилируется из исходников) и Tauri. Нет `sympy` — `pip install sympy`.

Критерий: все команды выполнены, недостающее установлено.

2. Сборка и первый запуск приложения

```
        # один раз: папка сборки Rust на диске с местом (3–6 ГБ)
        [Environment]::SetEnvironmentVariable('CARGO_TARGET_DIR', 'D:\AI\build\codewhale-desktop-target', 'User')
        # открыть НОВОЕ окно PowerShell после этого
        cd codewhale-desktop
        npm ci
        npm i -g codewhale          # агент; либо укажи путь к codewhale.exe в настройках приложения
        npm run tauri dev           # первая сборка Rust 10–20 минут
```

Ожидается окно «CodeWhale Desktop»: статус «Запуск CodeWhale» → чат. Если runtime не стартует — журнал: палитра `Ctrl+K` → «Журнал CodeWhale»; ошибки сборки — чинить в коде и коммитить.

Критерий: приложение открывается, в заголовке видна модель Ollama, простой запрос в чате получает ответ.

3. Настройки приложения

Настройки (`Ctrl+,`):

- **Ollama → Модель по умолчанию**: модель с инструментами и контекстом ≥ 16k (панель «Модели Ollama» умеет создать вариант с нужным `num_ctx`; контекст 4k по умолчанию — ловушка).
- **Диск знаний и Godot**: папка диска (у пользователя `F:\AI_Drive`; в примерах ниже `D:\AI_Drive`), путь к `godot.exe`.
- **Агент → автоподтверждение**: правило **ai-drive** включено по умолчанию (иначе каждый поиск по диску просит подтверждение).

Критерий: настройки сохранены (файл `%APPDATA%\codewhale-desktop\settings.json`).

4. Диск знаний

Корень диска знаний — любая папка на большом диске. У пользователя это **`F:\AI_Drive`** (рядом с `F:\PROJECTS\godot-hd`); пошаговый план именно для `F:` с размерами, ключами скрипта и проверками — `docs/KNOWLEDGE_DRIVE.md`, раздел «Диск F: как диск знаний». В примерах ниже вместо `D:\AI_Drive` подставляй `F:\AI_Drive`.

Панель `Ctrl+K` → «Диск знаний»:

1. **Подготовить диск** → в `<диск>\bin` появятся `sympy_helper.py` и `init-ai-drive.ps1`.
2. Наполнение (долго, можно частями):

   ```
   powershell -NoProfile -ExecutionPolicy Bypass -File D:\AI_Drive\bin\init-ai-drive.ps1 -DriveRoot D:\AI_Drive -SkipZim
   ```

   `-SkipZim` на первом проходе; дампы Kiwix (-Math, -Full) — отдельно, когда будет время. Доксеты Zeal: установить Zeal, скачать Godot/C++/Python/GLSL, скопировать папки `*.docset` из `%APPDATA%\Zeal\Zeal\docsets` в `D:\AI_Drive\docsets`.
3. **OLLAMA_MODELS → диск → Перезапустить Ollama** (если модели нужны на внешнем диске; иначе пропустить).
4. **Построить индекс** (после клонирования репозиториев в `git_vault`).
5. **Запустить Kiwix** (если есть `.zim` и `kiwix-serve.exe` в `bin`).
6. **Установить скиллы** → `~/.codewhale/skills/knowledge-drive`, `math-physics-godot`.
7. **Подключить MCP** → запись `ai-drive` в `%USERPROFILE%\.codewhale\mcp.json`, CodeWhale перезапустится.

Проверка из консоли (путь к exe — из панели, строка «MCP-сервер»):

```
& "D:\AI\build\codewhale-desktop-target\debug\codewhale-desktop.exe" mcp-drive status
& "...\codewhale-desktop.exe" mcp-drive call docset_search '{"query":"Vector3.slerp"}'
```

```
        & "...\codewhale-desktop.exe" mcp-drive call godot_check '{"path":"<абсолютный путь>\\Godot\\WaterDemo\\water.gdshader"}'
```

Критерий: `status` показывает разделы, `godot_check` возвращает `"ok": true, "uniforms": 11`; в чате запрос «найди в документации CharacterBody3D.move_and_slide» приводит к вызовам `tool_search` → `mcp_ai--drive_docset_search` (видно в карточках инструментов).

5. Финальный тест: шейдер воды с настоящей моделью

По `docs/FINAL_TEST_WATER.md`, раздел «Как повторить у себя»: открыть проект `Godot/WaterDemo` в приложении, отправить промпт из документа. Ожидаемый ход: агент подгружает скилл, пишет `water.gdshader`, вызывает `godot_check`, правит до `ok: true`, вызывает `ledger_record`. Затем рендер:

```
        & "F:\PROJECTS\godot-hd\engine\bin\godot.windows.editor.x86_64.exe" --path .\Godot\WaterDemo --resolution 1280x720 --quit-after 120 -- --shot D:\water.png
```

Критерий: `D:\water.png` похож на `docs/final-test/water.png`; запись в `D:\AI_Drive\.agent_state\ledger.json`. Если модель не справляется (не вызывает инструменты, путает имена) — зафиксировать, какая модель и что именно пошло не так, попробовать `qwen2.5-coder:7b/14b` с `num_ctx` 32k.

6. ComfyUI: фабрика ассетов

По `docs/COMFYUI.md`: папка ComfyUI с `main.py`, чекпоинт в `D:\AI_Drive\AI_ComfyModels\checkpoints` или `ComfyUI\models\checkpoints`, панель «Фабрика ассетов» → **Установить шаблоны** → выбрать чекпоинт → **Запустить ComfyUI** → **Подключить MCP**.

Проверка из консоли (без агента):

```
        $env:COMFY_URL='http://127.0.0.1:8188'; $env:AI_WORKFLOWS_DIR='D:\AI_Drive\AI_Workflows'; $env:COMFY_CHECKPOINT='<файл чекпоинта>'
        & "...\codewhale-desktop.exe" mcp-comfy call comfy_template_validate '{"template":"ui_icon"}'
        & "...\codewhale-desktop.exe" mcp-comfy call generate_ui_icon '{"prompt":"iron sword","save_to":"D:\\sword.png","size":512}'
```

Во время рендера в панели должен смениться индикатор «VRAM: рендер в ComfyUI (шаг N/M)» и появиться предпросмотр; `nvidia-smi` покажет, что модель Ollama выгружена. Затем в чате: «сгенерируй иконку меча в `res://assets/ui/icons/sword.png`».

Критерий: файл создан, `freedVram: true`, после генерации модель Ollama снова отвечает без долгой паузы. Если ComfyUI падает по памяти — уменьшить `size`, проверить `--reserve-vram`, записать модель/чекпоинт/лог.

7. Установщик и portable

Релиз **v0.1.0** уже опубликован из облачной сессии (Actions → Release, ручной запуск с полем `publish_tag`; тег создан workflow'ом на коммите `1ac6ac0`): https://github.com/Gepard1999sf/LOCAL_AGENT_PROJECT/releases/tag/v0.1.0

Файл	Размер	SHA-256
`CodeWhale-Desktop_0.1.0_x64-setup.exe` (NSIS, без прав администратора)	35,7 МБ	`cd741d85…e8e00f867`
`CodeWhale-Desktop_0.1.0_x64.msi` (для всех пользователей)	50,6 МБ	`cd471830…38bf3cfe3c`
`CodeWhale-Desktop_0.1.0_x64-portable.zip` (без установки: `codewhale-desktop.exe` + `codewhale-app-server.exe` + `resources\`)	52,1 МБ	`de5bc2bc…0ae53ea0`

Полные суммы видны на странице релиза (GitHub показывает `digest` у каждого файла) и проверяются так: `Get-FileHash .\CodeWhale-Desktop_0.1.0_x64-setup.exe -Algorithm SHA256`.

Что осталось проверить на локальной машине (в контейнере Windows нет):

1. `setup.exe` ставится без прав администратора, приложение запускается, SmartScreen обходится через «Подробнее → Выполнить в любом случае»; WebView2 при отсутствии докачивается.
2. Portable-zip распаковывается, `codewhale-desktop.exe` запускается (папку не разделять: агент `codewhale-app-server.exe` ищется рядом); настройки и журналы — в `%APPDATA%\net.codewhale.desktop`.
3. В обоих вариантах встроенный агент CodeWhale 0.10.1 (`codewhale-app-server.exe`) стартует (статус-точка в левом нижнем углу зелёная), `Ctrl+K` → «Диск знаний» и «Фабрика ассетов» открываются.

Следующий релиз: Actions → Release → **Run workflow** на нужной ветке, в поле `publish_tag` указать `v0.2.0` (или локально `git tag v0.2.0; git push origin v0.2.0`). Локальная сборка без релиза:

```
cd codewhale-desktop
npm run bundle    # NSIS + MSI в <CARGO_TARGET_DIR>\release\bundle\
```

8. Прочие хвосты (после основного)

Сделано в облаке 2026-10-16 (ветка `claude/busy-darwin-juot7g`, не переделывать):

- `ModelManager` (загрузка с прогрессом, удаление, копирование, варианты с `num_ctx`, оценка VRAM, вкладка «Запущенные») подключён в оверлей «Модели Ollama» вместо старой `ModelsPanel` (`src/components/overlays/SidePanels.tsx`). Моки `models_*` — `e2e/mock/backend.ts` и `e2e/mock/defaults.ts` (прогресс `models_pull` идёт в `Channel` страницы), 8 тестов — `e2e/tests/models.spec.ts`; весь набор e2e (41 тест) зелёный под Linux.
- `tools\check-shader.ps1` использует проект `codewhale-desktop\src-tauri\resources\godot-check` из репозитория (другой путь к Godot: `-Godot <exe>` или переменная `GODOT_BIN`).
- `release.yml` прошёл ревью: отказ публиковать `publish_tag`, который уже указывает на другой коммит; одна объединённая `--config` (версия из тега + sidecar) вместо двух, чтобы tauri-action подставлял версию тега в имя релиза и файлов; предупреждение, если `tauri.conf.json` не совпадает с тегом; ссылка на `INSTALL.md` по тегу; `GH_REPO` через env. Запуском не проверялось — смотреть при релизе v0.2.0.
- Синтез исследований: `docs/research/codewhale-safety-design.md`, `docs/research/rtx5060-8gb-model-plan.md`, `docs/research/user-pdf-verification.md`; ссылки — `PROJECT.md`, раздел 7.

Заготовка отчёта — `docs/LOCAL_RUN_REPORT.md`.

Осталось на машине пользователя:

- e2e на Windows: `cd codewhale-desktop\e2e; npm ci; npx playwright install chromium; npx playwright test` (ожидается 41 passed).
- `powershell -NoProfile -File tools\check-shader.ps1 Godot\WaterDemo\water.gdshader` → `COMPILES: ... (11 uniforms)`.
- Проверки из `docs/research/user-pdf-verification.md`, раздел 5 (скорость MoE на 8 ГБ, код возврата `--check-only`, эффект `LLAMA_ARG_FIT_TARGET` при открытом Godot и др.) и чек-лист приёмки модели из `docs/research/rtx5060-8gb-model-plan.md`, раздел 6.
- Контейнерные находки, которые стоит учитывать: MCP-инструменты в CodeWhale отложенные (`tool_search` обязателен), имена видны модели как `mcp_ai--drive_*`/`mcp_comfy_*`; `readOnlyHint` не снимает подтверждение для пользовательских серверов; в Compatibility-рендере Godot глубина из `hint_depth_texture` требует `depth * 2.0 - 1.0`.

9. Как отчитываться

В конце — короткий отчёт в `docs/LOCAL_RUN_REPORT.md` (заготовка уже в репозитории: таблица разделов 1–7 с «готово / не готово / почему», версии (Windows, Node, Rust, Ollama, модель, ComfyUI, Godot), что поменяно в коде). Коммиты — в ту же ветку `claude/busy-darwin-juot7g`, небольшими осмысленными порциями, с проверкой `npx tsc --noEmit` и `cargo clippy -- -D warnings` (в `src-tauri`) перед каждым.